



Tesina finale di
Signal Processing and Optimization for Big Data

Corso di Laurea Magistrale in Ingegneria Informatica e Robotica
Data Science and Data Engineering
A.A. 2025-2026

Docente:
Paolo Banelli

**Progettazione e analisi comparativa di
filtri polinomiali ai minimi quadrati per
il denoising di segnali su grafo**

Studiante:
Mencacci Giorgio
Matricola: 382814

Indice

1	Introduzione e motivazione	2
2	Costruzione del grafo	2
3	Modello di segnale e rumore sui nodi	2
3.1	L'assunzione di smoothness	2
3.1.1	Frequenza spettrale e total variation del laplaciano	2
3.1.2	Estrazione e ordinamento della base $\mathbf{U}$	3
3.1.3	Generazione del segnale pulito	3
3.2	Rumore AWGN	4
4	Filtri polinomiali su grafo	5
4.1	L'operatore di shift (GSO)	5
4.2	Dal filtraggio spettrale alla filtering matrix sui nodi	5
4.3	Motivazione dell'espansione polinomiale in $\mathbf{L}$	5
5	Strategie di progetto dei coefficienti	6
5.1	Metodo 1: least squares (LS) spettrale "Oracolo"	6
5.1.1	Campionamento e vettore target	7
5.1.2	Costruzione della matrice di Vandermonde	7
5.1.3	Soluzione problema LS regolarizzato	7
5.2	Metodo 2: LS spettrale da energia cumulativa	7
5.2.1	Implementazione	8
5.3	Metodo 3 Data-Driven: apprendimento supervisionato della risposta del filtro	8
5.3.1	Declinazione del framework di System Identification al denoising . .	8
5.3.2	Costruzione della matrice $\mathbf{H}$ e derivazione analitica	9
5.3.3	Procedura di addestramento e stima dei coefficienti	10
5.3.4	Implementazione	11
5.4	Implementazione dell'operazione di filtraggio	11
5.4.1	Implementazione	12
6	Denoising spettrale basato su stima empirica dell'energia	12
6.1	Risposta sui singoli vertici e analisi spettrale	12
6.1.1	Analisi di robustezza tramite grid search	14
7	Denoising guidato dai dati (Data-Driven System Identification)	16
7.1	Risposta sui singoli vertici e analisi spettrale	16
7.1.1	Analisi di generalizzazione al variare di K e SNR_{in}	17
8	Riproducibilità	19

1 Introduzione e motivazione

Negli ultimi anni, l'esplosione dei dati generati da infrastrutture complesse e distribuite ha spostato l'attenzione del *Signal Processing* dai domini euclidei regolari a domini geometrici irregolari. In questi contesti, la struttura relazionale dei dati è formalizzata in modo naturale mediante un **grafo** $G = (\mathcal{V}, \mathcal{E})$, dove $\mathcal{V}$ rappresenta l'insieme degli N nodi (vertici) e $\mathcal{E}$ definisce le interazioni, pesate o binarie, tra le coppie di nodi.

2 Costruzione del grafo

I nodi sono posizionati campionando uniformemente le coordinate su un dominio planare $[0, 100] \times [0, 100]$:

$$\mathbf{p}_i = (x_i, y_i) \sim \mathcal{U}([0, 100]^2), \quad \forall i = 1, \dots, N \quad (1)$$

La connettività locale è determinata mediante il paradigma dei k -Nearest Neighbors (k -NN con $k = 4$), implementato tramite la libreria spaziale `libpysal`. Poiché il vicinato a k -NN definisce un grafo diretto asimmetrico (se il nodo j è tra i 4 più vicini a i , non è garantito il viceversa), la matrice di adiacenza binaria $\mathbf{A} \in \{0, 1\}^{N \times N}$ viene resa simmetrica e priva di auto-anelli:

$$\mathbf{A}_{\text{sym}} = \max(\mathbf{A}_{k\text{-NN}}, \mathbf{A}_{k\text{-NN}}^T), \quad A_{ii} = 0 \quad (2)$$

Tale disposizione modella realisticamente lo scenario di una rete di sensori distribuiti, in cui ciascuna unità campiona grandezze fisiche locali sul territorio e scambia informazioni a corto raggio.

3 Modello di segnale e rumore sui nodi

3.1 L'assunzione di smoothness

Il fondamento teorico del filtraggio su grafo risiede nell'ipotesi che i segnali fisici siano **smooth** (ovvero a variazione spaziale lenta) rispetto alla topologia della rete. In termini intuitivi, nodi fortemente connessi nel grafo tendono ad assumere valori simili. Dal punto di vista della trasformata di Fourier su grafo (GFT), ciò implica che il segnale deterministico originale è a **banda limitata**, concentrando l'intera propria energia sui modi spettrali a bassa frequenza (corrispondenti agli autovalori più piccoli della matrice Laplaciana $\mathbf{L}$).

3.1.1 Frequenza spettrale e total variation del laplaciano

La corrispondenza teorica tra l'ampiezza degli autovalori λ_i e il concetto di frequenza spaziale discende direttamente dalla **Total Variation**. Per un generico segnale su grafo $\mathbf{x} \in \mathbb{R}^N$, la variazione spaziale tra nodi adiacenti è formalizzata dalla forma quadratica associata alla matrice Laplaciana:

$$\text{TV}(\mathbf{x}) = \mathbf{x}^T \mathbf{L} \mathbf{x} = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N A_{ij} (x_i - x_j)^2 \quad (3)$$

Per comprendere il significato fisico delle singole componenti spettrali, si può assumere come segnale sui vertici l'autovettore k -esimo del Laplaciano $\mathbf{u}_k$, interpretando ciascuna componente $u_k(i)$ come il valore assunto sul nodo i -esimo. Sostituendo $\mathbf{u}_k$ all'interno della forma quadratica e sfruttando l'equazione agli autovalori $\mathbf{L}\mathbf{u}_k = \lambda_k\mathbf{u}_k$ unita all'ortonormalità della base ($\|\mathbf{u}_k\|_2^2 = \mathbf{u}_k^T\mathbf{u}_k = 1$), si ottiene la catena di uguaglianze:

$$\text{TV}(\mathbf{u}_k) = \mathbf{u}_k^T \mathbf{L} \mathbf{u}_k = \mathbf{u}_k^T (\lambda_k \mathbf{u}_k) = \lambda_k (\mathbf{u}_k^T \mathbf{u}_k) = \lambda_k \quad (4)$$

Emerge chiaramente che l'autovalore λ_k non è un mero indice algebrico, ma coincide esattamente con la misura quantitativa della variazione locale dell'autovettore $\mathbf{u}_k$ lungo gli archi del grafo:

- **Basse frequenze** ($\lambda_k \approx 0$): Poiché la somma delle differenze al quadrato deve risultare quasi nulla, la discrepanza $(u_k(i) - u_k(j))^2$ tra nodi connessi ($A_{ij} = 1$) è minima. L'autovettore $\mathbf{u}_k$ descrive quindi una funzione lenta e globalmente liscia (*smooth*) sulla rete.
- **Alte frequenze** ($\lambda_k \rightarrow 2$): Il valore elevato impone differenze marcate tra nodi adiacenti. L'autovettore $\mathbf{u}_k$ assume un andamento fortemente oscillatorio, invertendo repentinamente segno e ampiezza tra vertici vicini.

3.1.2 Estrazione e ordinamento della base $\mathbf{U}$

Per garantire che la base di Fourier su grafo rifletta la gerarchia naturale delle frequenze, la decomposizione spettrale avviene sulla matrice $\mathbf{L}_{\text{norm}} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ (dopo aver generato il grafo vedi sezione 2) che viene seguita da una fase di riordinamento. La diagonalizzazione tramite `np.linalg.eigh` produce gli autovalori e la matrice degli autovettori organizzati per colonna. Tramite la determinazione della permutazione degli indici ordinati $\pi = \text{argsort}(\mathbf{\Lambda})$, lo spettro e la base ortonormale vengono allineati in modo crescente:

$$0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{N-1} \leq 2, \quad \mathbf{U} = [\mathbf{u}_{\pi(0)} \quad \mathbf{u}_{\pi(1)} \quad \dots \quad \mathbf{u}_{\pi(N-1)}] \quad (5)$$

```

1 def compute_fourier_basis(self):
2     """Calcola autovalori e autovettori di L_norm (Base GFT)."""
3     eigenvalues, eigenvectors = np.linalg.eigh(self.L_norm_sparse.
4         toarray())
5     idx = np.argsort(eigenvalues)
6     self.eigenvalues = eigenvalues[idx]
7     self.U = eigenvectors[:, idx]
8     return self.eigenvalues, self.U

```

3.1.3 Generazione del segnale pulito

Per generare sinteticamente un segnale esattamente *smooth*, si è operato nel dominio trasformato tramite la trasformata inversa di Fourier su grafo (iGFT). Dalla decomposizione spettrale della matrice Laplaciana reale e simmetrica $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$, le colonne di $\mathbf{U} = [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1}]$ costituiscono una **base ortonormale**. Grazie all'ortonormalità di $\mathbf{U}$, qualsiasi segnale $\mathbf{x} \in \mathbb{R}^N$ ammette una rappresentazione spettrale unica data dalla combinazione lineare $\mathbf{x} = \mathbf{U}\hat{\mathbf{x}} = \sum_{i=0}^{N-1} \hat{x}_i \mathbf{u}_i$. Un segnale è strettamente a banda limitata entro i primi C modi spettrali ($C = \text{cutoff_idx}$) se i suoi coefficienti soddisfano $\hat{x}_i = 0$ per ogni $i \geq C$.

La procedura adottata per generare il segnale pulito si articola nei seguenti passaggi:

1. Campionamento del vettore dei coefficienti spettrali per le sole prime C frequenze:

$$\mathbf{c} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (6)$$

ponendo a zero tutte le restanti componenti ad alta frequenza ($\hat{x}_i = 0$ per ogni $i \geq C$);

2. Estrazione della base ridotta $\mathbf{U}_C \in \mathbb{R}^{N \times C}$, costituita dalle prime C colonne ortonormali della matrice $\mathbf{U}$. Poiché gli autovalori del Laplaciano sono ordinati in modo crescente ($0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{N-1}$), i primi C autovettori $\mathbf{u}_0, \dots, \mathbf{u}_{C-1}$ corrispondono alle frequenze più basse, ovvero ai modi fondamentali che descrivono segnali globalmente lisci (*smooth*) sulla topologia della rete;
3. Sintesi del segnale nel dominio dei vertici mediante iGFT ristretta al sottospazio di banda base:

$$\mathbf{x}_{\text{smooth}} = \mathbf{U}_C \mathbf{c} = \sum_{i=0}^{C-1} c_i \mathbf{u}_i \quad (7)$$

L'implementazione avviene in Python come segue:

```

1 # 1. Coefficienti spettrali sulle prime 'cutoff_idx' frequenze
2 c = np.random.randn(cutoff_idx)
3
4 # 2. Ricostruzione nello spazio dei nodi tramite iGFT ridotta: x = U_C @
   c
5 x_smooth = self.topology.U[:, :cutoff_idx] @ c

```

3.2 Rumore AWGN

In uno scenario reale, le misurazioni acquisite dai nodi sono degradate da disturbi. Il rumore è formalizzato tramite il modello a **Rumore Bianco Gaussiano Additivo (AWGN)**:

$$\mathbf{x}_\varepsilon = \mathbf{x} + \varepsilon \quad (8)$$

dove:

- $\mathbf{x} \in \mathbb{R}^N$ è il segnale deterministico originale a banda limitata;
- $\varepsilon \in \mathbb{R}^N$ è il vettore di rumore casuale a elementi indipendenti e identicamente distribuiti (i.i.d.):

$$\varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}) \quad (9)$$

caratterizzato da media nulla $\mathbb{E}[\varepsilon] = \mathbf{0}$ e matrice di covarianza $\mathbb{E}[\varepsilon \varepsilon^T] = \sigma^2 \mathbf{I}$. Poiché il rumore ha media nulla ($\mathbb{E}[\varepsilon_i] = 0$), la sua potenza coincide con la varianza:

$$P_{\text{noise}} = \mathbb{E}[\varepsilon_i^2] = \text{Var}(\varepsilon_i) + (\mathbb{E}[\varepsilon_i])^2 = \sigma^2 \quad (10)$$

Il rapporto segnale-rumore in ingresso quantifica la qualità dell'osservazione rispetto alla potenza media del segnale pulito:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_s}{\sigma^2} \right) \quad (11)$$

Per impostare un valore di SNR_{dB} target nelle simulazioni, la varianza richiesta viene calcolata invertendo la relazione precedente:

$$\sigma^2 = \frac{P_s}{10^{\frac{\text{SNR}_{\text{dB}}}{10}}} \quad (12)$$

I campioni casuali $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ vengono quindi calcolati e sommati a ciascun nodo per ottenere il vettore di test corrotto $\mathbf{x}_{\text{noisy}}$.

4 Filtri polinomiali su grafo

4.1 L'operatore di shift (GSO)

L'elemento cardine del *Graph Signal Processing* è l'operatore di shift (*Graph Shift Operator*, GSO), una matrice $\mathbf{S} \in \mathbb{R}^{N \times N}$ che cattura la struttura locale del grafo. In questa tesina, è stato adottato come GSO il **Laplaciano Normalizzato**, definito come:

$$\mathcal{L}_{\text{norm}} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2} \quad (13)$$

dove $\mathbf{A}$ è la matrice di adiacenza e $\mathbf{D}$ è la matrice dei gradi.

4.2 Dal filtraggio spettrale alla filtering matrix sui nodi

Nel dominio spettrale della GFT, l'azione di un filtro con risposta desiderata $\hat{\mathbf{h}} = [\hat{h}_1, \dots, \hat{h}_N]^T$ applicato al segnale trasformato $\hat{\mathbf{x}}$ è espressa tramite il prodotto elemento per elemento:

$$\hat{\mathbf{y}} = \text{diag}(\hat{\mathbf{h}}) \hat{\mathbf{x}} \quad (14)$$

Riportando l'uscita nel dominio dei nodi tramite la trasformata inversa di Fourier su grafo e sostituendo la definizione diretta $\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}$, si ottiene:

$$\mathbf{y} = \mathbf{U} \text{diag}(\hat{\mathbf{h}}) \mathbf{U}^T \mathbf{x} = \mathbf{H} \mathbf{x} \quad (15)$$

dove $\mathbf{H} = \mathbf{U} \text{diag}(\hat{\mathbf{h}}) \mathbf{U}^T \in \mathbb{R}^{N \times N}$ rappresenta la **Filtering Matrix** nel dominio dei vertici. Tale formulazione evidenzia un collo di bottiglia computazionale critico: il calcolo esplicito di $\mathbf{y}$ dipende dalla matrice ortonormale $\mathbf{U}$, la quale richiede la decomposizione spettrale completa del Laplaciano ($\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$). L'*eigenvector decomposition* è un'operazione a complessità cubica $\mathcal{O}(N^3)$, che risulta computazionalmente proibitiva all'aumentare della dimensione del grafo.

4.3 Motivazione dell'espansione polinomiale in $\mathbf{L}$

Per superare tale limitazione, si osserva che la matrice $\mathbf{H} = \mathbf{U} \text{diag}(\hat{\mathbf{h}}) \mathbf{U}^T$ ha le colonne appartenenti a $\text{RangeSpace}(\mathbf{U})$, che coincide esattamente con lo spazio delle colonne della matrice Laplaciana:

$$\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T = \mathbf{U} \text{diag}([\lambda_1, \dots, \lambda_N]) \mathbf{U}^T \quad (16)$$

Esiste dunque una stretta relazione algebrica che consente di esprimere $\mathbf{H}$ come funzione diretta dell'operatore $\mathbf{L}$, evitando di passare attraverso il calcolo esplicito degli autovettori in $\mathbf{U}$. Facendo un parallelo con il filtraggio classico 1D nel dominio continuo delle frequenze, dove una risposta generica può essere sviluppata in serie polinomiale $H(f) = \sum \alpha_n f^n$,

nel contesto del Graph Signal Processing gli autovalori λ_k svolgono il ruolo delle frequenze discrete. La risposta in frequenza $\hat{h}_k = h(\lambda_k)$ può essere approssimata come espansione polinomiale di grado K :

$$\hat{h}_k = h(\lambda_k) \approx \sum_{n=0}^K \alpha_n \lambda_k^n, \quad \forall k = 1, \dots, N \quad (17)$$

Poiché gli autovalori λ_k sono contenuti nella matrice diagonale $\mathbf{\Lambda}$, sfruttando la proprietà di diagonalizzazione delle potenze di matrice:

$$\mathbf{L}^n = \underbrace{\mathbf{L} \cdot \mathbf{L} \cdots \mathbf{L}}_{n \text{ volte}} = (\mathbf{U}\mathbf{\Lambda}\mathbf{U}^T) \cdots (\mathbf{U}\mathbf{\Lambda}\mathbf{U}^T) = \mathbf{U}\mathbf{\Lambda}^n\mathbf{U}^T = \mathbf{U} \text{diag}([\lambda_1^n, \dots, \lambda_N^n])\mathbf{U}^T \quad (18)$$

l'operatore di filtraggio $\mathbf{H}$ può essere approssimato direttamente tramite una combinazione lineare di potenze del Laplaciano:

$$\mathbf{H} \approx \sum_{n=0}^K \alpha_n \mathbf{L}^n = \mathbf{U} \underbrace{\left(\sum_{n=0}^K \alpha_n \mathbf{\Lambda}^n \right)}_{\approx \text{diag}(\hat{\mathbf{h}})} \mathbf{U}^T \quad (19)$$

dove la matrice diagonale interna rappresenta l'approssimazione polinomiale della risposta spettrale desiderata $\hat{\mathbf{h}}$. L'applicazione del filtro al segnale sui nodi si riduce pertanto a:

$$\mathbf{y} = \mathbf{H}\mathbf{x} = \sum_{n=0}^K \alpha_n \mathbf{L}^n \mathbf{x} = \alpha_0 \mathbf{x} + \alpha_1 \mathbf{L}\mathbf{x} + \cdots + \alpha_K \mathbf{L}^K \mathbf{x} \quad (20)$$

Questa formulazione comporta come vantaggio architetturale fondamentale la possibilità di calcolare l'uscita $\mathbf{y}$ senza mai costruire esplicitamente la matrice $\mathbf{H}$, evitando così il costo computazionale proibitivo della decomposizione spettrale completa. Invece, l'operazione si riduce a una sequenza di moltiplicazioni **ricorsive** matrice-vettore $\mathbf{L}^n \mathbf{x}$.

Il problema di *Filter Design* si riconduce quindi alla determinazione dei soli $K + 1$ coefficienti ottimi $\{\alpha_n\}_{n=0}^K$ capaci di adattare al meglio la risposta target.

5 Strategie di progetto dei coefficienti

5.1 Metodo 1: least squares (LS) spettrale "Oracolo"

Si assume di avere una conoscenza perfetta (oracolo) della frequenza di taglio ideale λ_c del segnale originale. L'obiettivo è approssimare una risposta in frequenza ideale passa-basso:

$$g(\lambda) = \begin{cases} 1 & \text{se } \lambda \leq \lambda_c \\ 0 & \text{se } \lambda > \lambda_c \end{cases} \quad (21)$$

Il problema viene risolto tramite una regressione ai minimi quadrati sulla **matrice di Vandermonde** costruita sugli autovalori del grafo $\lambda_1, \dots, \lambda_N$:

$$\min_{\mathbf{h}} \|\mathbf{V}\mathbf{h} - \mathbf{g}\|_2^2 + \mu \|\mathbf{h}\|_2^2 \quad (22)$$

dove $V_{i,k} = \lambda_i^k$. La regolarizzazione di Tikhonov (μ) garantisce la stabilità numerica.

5.1.1 Campionamento e vettore target

Per prima cosa, la risposta in frequenza desiderata $g(\lambda)$ viene campionata in corrispondenza degli autovalori del grafo $\lambda_1, \dots, \lambda_N$ precedentemente calcolati. Questo definisce il vettore target $\mathbf{g} \in \mathbb{R}^N$:

$$\mathbf{g} = [g(\lambda_1), g(\lambda_2), \dots, g(\lambda_N)]^T \quad (23)$$

Nel codice, questo passaggio è implementato tramite una *list comprehension*:

```
1 def ideal_lowpass_oracle(lam):
2     return 1.0 if lam <= lambda_cutoff_oracle else 0.0
3
4 g_target = np.array([ideal_lowpass_oracle(lam) for lam in lambdas])
```

5.1.2 Costruzione della matrice di Vandermonde

Un filtro polinomiale di ordine K applicato nel dominio spettrale agisce come $H(\lambda_i) = \sum_{k=0}^K h_k \lambda_i^k$. Per rappresentare simultaneamente la risposta del filtro su tutti gli autovalori, si costruisce la matrice $\mathbf{V} \in \mathbb{R}^{N \times (K+1)}$:

$$\mathbf{V} = \begin{bmatrix} 1 & \lambda_1 & \lambda_1^2 & \dots & \lambda_1^K \\ 1 & \lambda_2 & \lambda_2^2 & \dots & \lambda_2^K \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \lambda_N & \lambda_N^2 & \dots & \lambda_N^K \end{bmatrix} \quad (24)$$

```
1 lambdas = self.topology.eigenvalues
2 V = np.column_stack([lambdas**k for k in range(self.K + 1)])
```

5.1.3 Soluzione problema LS regolarizzato

Il problema di trovare i coefficienti ottimali $\mathbf{h} = [h_0, \dots, h_K]^T$ viene formulato come un problema di **Ridge Regression** per minimizzare l'errore quadratico tra la risposta del polinomio e il target ideale:

$$J(\mathbf{h}) = \|\mathbf{V}\mathbf{h} - \mathbf{g}\|_2^2 + \mu \|\mathbf{h}\|_2^2 \quad (25)$$

La soluzione ottima si ottiene risolvendo l'equazione derivata dal gradiente nullo:

$$(\mathbf{V}^T \mathbf{V} + \mu \mathbf{I}) \mathbf{h} = \mathbf{V}^T \mathbf{g} \quad (26)$$

L'implementazione traduce questa equazione utilizzando il solutore lineare di *NumPy*:

```
1 I_reg = np.eye(self.K + 1)
2 A_ls = V.T @ V + self.mu * I_reg
3 b_ls = V.T @ g_target
4 self.h = np.linalg.solve(A_ls, b_ls)
```

5.2 Metodo 2: LS spettrale da energia cumulativa

In scenari reali, la banda del segnale pulito non è nota a priori. Questo metodo stima empiricamente λ_c analizzando lo spettro del segnale rumoroso $\mathbf{x}_{\text{noisy}}$. Si calcola la **curva dell'energia spettrale cumulativa**:

$$E(i) = \frac{\sum_{j=1}^i \hat{x}_j^2}{\sum_{j=1}^N \hat{x}_j^2} \quad (27)$$

Si identifica l'indice i^* tale per cui $E(i^*) \geq \tau$ (dove τ è una soglia, ad esempio 0.75). La frequenza di taglio stimata sarà $\lambda_{\text{energy}} = \lambda_{i^*}$. Una volta ottenuta questa stima, si procede al fit spettrale come nel Metodo 1. Questa strategia è sensibile alla scelta della soglia τ .

5.2.1 Implementazione

Nel codice, la determinazione empirica della frequenza di taglio è implementata in modo vettorizzato sfruttando le primitive di *NumPy*:

```

1 # 1. Calcolo dell'energia spettrale per ciascun autovalore: |x_hat_i|^2
2 gft_energy = (g.U.T @ x_noisy) ** 2
3
4 # 2. Curva dell'energia cumulativa normalizzata in [0, 1]
5 cum_energy = np.cumsum(gft_energy) / np.sum(gft_energy)
6
7 # 3. Ricerca binaria del primo indice che supera la soglia prefissata (
8     tau = 0.75)
9 cutoff_idx_energy = np.searchsorted(cum_energy, 0.75)
10
11 # 4. Frequenza di taglio stimata corrispondente all'autovalore i-esimo
12 lambda_cutoff_energy = g.eigenvalues[cutoff_idx_energy]
```

La funzione `np.cumsum` calcola la somma progressiva delle energie spettrali $\sum_{j=1}^i |\hat{x}_j|^2$, mentre `np.searchsorted` esegue una ricerca binaria determinando istantaneamente il più piccolo indice i^* tale per cui l'energia cumulata raggiunga almeno la soglia specificata ($\tau = 75\%$). L'autovalore λ_{i^*} associato viene quindi impiegato per parametrizzare la risposta ideale $g(\lambda)$.

Nota metodologica: La specifica formulazione algoritmica di questo stimatore euristico è stata sviluppata con il supporto di un modello di intelligenza artificiale (Gemini, Google).

5.3 Metodo 3 Data-Driven: apprendimento supervisionato della risposta del filtro

Nei metodi spettrali precedenti la progettazione del filtro richiede di definire a priori una funzione di trasferimento ideale $g(\lambda)$. Al contrario, in questo approccio l'obiettivo è determinare i coefficienti del filtro $\mathbf{h} = [h_0, h_1, \dots, h_K]^T$ forzando l'operatore a inseguire direttamente un profilo di riferimento pulito $\mathbf{x}_{\text{clean}}$, apprendendo la trasformazione ottimale direttamente dalle realizzazioni del segnale sui nodi.

5.3.1 Declinazione del framework di System Identification al denoising

Nel framework di *System Identification* su grafi [1], il problema è formulato come un modello di propagazione fisica diretta: si assume che un segnale di ingresso $\mathbf{x}_{\text{in}}$ (la sorgente non corrotta) diffonda attraverso la topologia del grafo secondo una dinamica lineare incognita $\mathbf{H}(\mathbf{h}, \mathbf{S})$, generando un'uscita $\mathbf{y}_{\text{target}}$ osservata con rumore:

$$\mathbf{y}_{\text{target}} = \mathbf{H}(\mathbf{h}, \mathbf{S})\mathbf{x}_{\text{in}} + \mathbf{n} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}_{\text{in}} + \mathbf{n}, \quad (28)$$

dove l'obiettivo standard consiste nello stimare i parametri $\mathbf{h} = [h_0, \dots, h_K]^T$ per caratterizzare il processo di diffusione o il mezzo di trasmissione sottostante. In questo lavoro,

tale paradigma viene traslato a un problema inverso di **regressione supervisionata per il filtraggio**: anziché modellare la dinamica che corrompe o diffonde il segnale, i ruoli delle variabili vengono invertiti per apprendere direttamente l'operatore di ripristino:

- **Ingresso del filtro** ($\mathbf{x}_{\text{in}} \leftarrow \mathbf{x}_{\text{noisy}}$): coincide con il segnale corrotto o degradato osservato sui nodi del grafo;
- **Target di riferimento** ($\mathbf{y}_{\text{target}} \leftarrow \mathbf{x}_{\text{clean}}$): rappresenta la controparte ideale priva di rumore (ground truth).

In questo modo, la procedura di stima non mira a diagnosticare il canale di degradazione, ma risponde direttamente alla domanda: *qual è la combinazione lineare ottima di diffusioni sui nodi ($\mathbf{S}^k \mathbf{x}_{\text{noisy}}$) capace di restituire il segnale pulito originale?*

Sotto questa impostazione, l'apprendimento dei coefficienti $\mathbf{h}^*$ equivale a stimare un operatore di regolarizzazione data-driven $\mathbf{H}(\mathbf{h}^*, \mathbf{S})$ che agisce come filtro di pulizia:

$$\mathbf{x}_{\text{clean}} \approx \sum_{k=0}^K h_k^* \mathbf{S}^k \mathbf{x}_{\text{noisy}}. \quad (29)$$

Questo schema offre come vantaggio principale la possibilità di apprendere automaticamente la risposta in frequenza ottimale del filtro direttamente dai dati, senza richiedere la progettazione manuale di una funzione di taglio o l'imposizione a priori di un comportamento passa-basso. In altre parole, il filtro si adatta ai pattern del segnale e del rumore presenti nel dataset di addestramento.

5.3.2 Costruzione della matrice $\mathbf{H}$ e derivazione analitica

Sfruttando la linearità nei parametri del filtro polinomiale $\mathbf{H}(\mathbf{h}, \mathbf{S}) = \sum_{k=0}^K h_k \mathbf{S}^k$, l'azione di filtraggio sul segnale di ingresso $\mathbf{x}_{\text{in}} \in \mathbb{R}^N$ può essere riscritta come il prodotto tra una matrice di diffusione e il vettore dei coefficienti $\mathbf{h} = [h_0, \dots, h_K]^\top \in \mathbb{R}^{K+1}$. Tale matrice coincide con la base del *sottospazio di Krylov* $\mathcal{K}_{K+1}(\mathbf{S}, \mathbf{x}_{\text{in}})$, definita come:

$$\mathbf{X} = [\mathbf{x}_{\text{in}}, \mathbf{S}\mathbf{x}_{\text{in}}, \mathbf{S}^2\mathbf{x}_{\text{in}}, \dots, \mathbf{S}^K\mathbf{x}_{\text{in}}] \in \mathbb{R}^{N \times (K+1)}, \quad (30)$$

```

1
2 def _build_krylov_matrix(self, x: np.ndarray) -> np.ndarray:
3
4     L = self.topology.L_norm_sparse
5     shifts = [x]
6     curr = x.copy()
7
8     for _ in range(self.K):
9         curr = L.dot(curr)
10        shifts.append(curr)
11
12    return np.column_stack(shifts)

```

Formulazione originaria nel framework GSP. Nella formulazione generale proposta da Isufi et al. [1], il problema di stima dei parametri considera un'osservazione parziale del target e promuove la sparsità spettrale/strutturale tramite ottimizzazione convessa non differenziabile:

$$\mathbf{h}^* = \arg \min_{\mathbf{h}} \|\mathbf{M}\mathbf{y}_{\text{target}} - \mathbf{M}\mathbf{X}\mathbf{h}\|_2^2 + \gamma \|\text{diag}(\boldsymbol{\omega})\mathbf{h}\|_1, \quad (31)$$

dove $\mathbf{M} \in \{0, 1\}^{M \times N}$ (con $M \leq N$) è la matrice di campionamento che seleziona i nodi osservati, $\boldsymbol{\omega} \in \mathbb{R}_+^{K+1}$ è un vettore di pesi crescenti per penalizzare i gradi elevati di $\mathbf{S}$, e il termine ℓ_1 costringe molti coefficienti ad azzerarsi. Tale impostazione richiede risolutori iterativi.

Semplificazione e soluzione in forma chiusa adottata. Nel presente lavoro, la formulazione (31) viene opportunamente riadattata introducendo due semplificazioni:

1. **Osservabilità globale ($\mathbf{M} = \mathbf{I}_N$):** Si assume l'accesso all'intero grafo in fase di addestramento ($M = N$), eliminando la matrice di selezione $\mathbf{M}$.
2. **Regolarizzazione ℓ_2 (Ridge/Tikhonov):** La penalità pesata ℓ_1 viene sostituita con una norma ℓ_2 euclidea $\|\mathbf{h}\|_2^2$ pesata dal fattore di regolarizzazione $\mu > 0$.

Il problema di identificazione si traduce quindi:

$$J(\mathbf{h}) = \|\mathbf{X}\mathbf{h} - \mathbf{y}_{\text{target}}\|_2^2 + \mu \|\mathbf{h}\|_2^2. \quad (32)$$

Annullando il gradiente $\nabla_{\mathbf{h}} J(\mathbf{h}) = 2\mathbf{X}^\top(\mathbf{X}\mathbf{h} - \mathbf{y}_{\text{target}}) + 2\mu\mathbf{h} = \mathbf{0}$, si ottiene la soluzione ottima analitica **in forma chiusa**:

$$\mathbf{h}^* = (\mathbf{X}^\top \mathbf{X} + \mu \mathbf{I}_{K+1})^{-1} \mathbf{X}^\top \mathbf{y}_{\text{target}}. \quad (33)$$

si ottiene il sistema delle equazioni normali regolarizzate:

$$(\mathbf{X}^\top \mathbf{X} + \mu \mathbf{I})\mathbf{h} = \mathbf{X}^\top \mathbf{y}_{\text{target}} \quad (34)$$

5.3.3 Procedura di addestramento e stima dei coefficienti

La calibrazione del filtro supervisionato si articola nei seguenti passaggi:

1. **Generazione della coppia di calibrazione:** Viene sintetizzato un segnale a banda limitata $\mathbf{x}_{\text{clean, train}} \in \mathbb{R}^N$ (con banda confinata ai primi $C = 5$ modi spettrali) e gli viene sommato un disturbo gaussiano bianco per ottenere l'osservazione degradata $\mathbf{x}_{\text{noisy, train}}$.
2. **Costruzione dello spazio di Krylov:** A partire dall'ingresso rumoroso, si calcolano iterativamente le diffusioni locali sul grafo tramite moltiplicazioni matrice sparsa-vettore $\mathbf{x}^{(k)} = \mathbf{L}\mathbf{x}^{(k-1)}$, assemblando la matrice di Krylov $\mathbf{X}_{\text{train}} \in \mathbb{R}^{N \times (K+1)}$:

$$\mathbf{X}_{\text{train}} = [\mathbf{x}_{\text{noisy, train}} \quad \mathbf{L}\mathbf{x}_{\text{noisy, train}} \quad \dots \quad \mathbf{L}^K \mathbf{x}_{\text{noisy, train}}] \quad (35)$$

3. **Ottimizzazione ai minimi quadrati regolarizzati:** I coefficienti ottimi $\mathbf{h}^* \in \mathbb{R}^{K+1}$ vengono identificati minimizzando l'errore quadratico rispetto al target pulito con penalizzazione di Tikhonov ($\mu = 10^{-3}$):

$$\mathbf{h}^* = \arg \min_{\mathbf{h}} \{\|\mathbf{X}_{\text{train}}\mathbf{h} - \mathbf{x}_{\text{clean, train}}\|_2^2 + \mu \|\mathbf{h}\|_2^2\} \quad (36)$$

I pesi $\mathbf{h}^*$ così determinati rappresentano la risposta di diffusione ottimale e vengono successivamente congelati per la fase di test.

Per garantire che il filtro stimato $\mathbf{h}^*$ non sia affetto da *overfitting* e che abbia effettivamente appreso un operatore di filtraggio generale per la classe di segnali considerata, la validazione viene condotta su un'istanza di test indipendente:

1. **Generazione dell'istanza di test:** Viene sintetizzata una nuova coppia di test indipendente ($\mathbf{x}_{\text{clean, test}}, \mathbf{x}_{\text{noisy, test}}$) condividendo la medesima banda spettrale ($C = 5$) della fase di calibrazione. L'indipendenza rispetto ai dati di addestramento è garantita impostando seeds disgiunti per il generatore pseudo-casuale:

$$\mathbf{x}_{\text{noisy, test}} = \mathbf{x}_{\text{clean, test}} + \mathbf{n}_{\text{test}}. \quad (37)$$

2. **Quantificazione della capacità di generalizzazione:** La fedeltà del segnale ricostruito $\hat{\mathbf{x}}_{\text{test}}$ viene valutata rispetto al target ideale $\mathbf{x}_{\text{clean, test}}$ calcolando l'errore quadratico medio (MSE) e il guadagno sul rapporto segnale-rumore (SNR_{out}):

$$\text{MSE}_{\text{test}} = \frac{1}{N} \|\hat{\mathbf{x}}_{\text{test}} - \mathbf{x}_{\text{clean, test}}\|_2^2, \quad \text{SNR}_{\text{out}} = 10 \log_{10} \left(\frac{\|\mathbf{x}_{\text{clean, test}}\|_2^2}{\|\hat{\mathbf{x}}_{\text{test}} - \mathbf{x}_{\text{clean, test}}\|_2^2} \right) \quad (38)$$

5.3.4 Implementazione

Nel codice sviluppato, la procedura viene eseguita all'interno del metodo `fit_data_driven`:

```

1 def fit_data_driven(self, x_in: np.ndarray, y_target: np.ndarray):
2     """
3     Metodo Data-Driven: Fitta i coefficienti h mappando l'ingresso
4     rumoroso
5     x_in nel segnale target pulito y_target risolvendo ||X*h - y||_2^2 +
6     mu*||h||_2^2.
7     """
8     # 1. Costruisce la matrice dei segnali shiftati: X = [x, L*x, L^2*x,
9     # ..., L^K*x]
10    X = self._build_krylov_matrix(x_in) # shape (N, K+1)
11
12    # 2. Risoluzione Ridge Regression: h = (X^T X + mu * I)^(-1) X^T y
13    I_reg = np.eye(self.K + 1)
14    A_ls = X.T @ X + self.mu * I_reg
15    b_ls = X.T @ y_target
16    self.h = np.linalg.solve(A_ls, b_ls)
17
18    return self.h

```

5.4 Implementazione dell'operazione di filtraggio

L'applicazione del filtro polinomiale sul segnale di grafo $\mathbf{x}$ corrisponde all'operazione lineare:

$$\mathbf{y} = \mathbf{H}(\mathbf{L})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{L}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{L} \mathbf{x} + h_2 \mathbf{L}^2 \mathbf{x} + \dots + h_K \mathbf{L}^K \mathbf{x} \quad (39)$$

Per ottimizzare il calcolo computazionale ed evitare l'elevamento a potenza esplicito della matrice sparsa $\mathbf{L}$, il segnale diffuso al salto k -esimo viene calcolato in modo ricorsivo:

$$\begin{cases} \mathbf{x}^{(0)} = \mathbf{x} \\ \mathbf{x}^{(k)} = \mathbf{L} \mathbf{x}^{(k-1)}, \quad \text{per } k = 1, \dots, K \end{cases} \quad (40)$$

consentendo di accumulare progressivamente l'uscita:

$$\mathbf{y} = \sum_{k=0}^K h_k \mathbf{x}^{(k)} = h_0 \mathbf{x}^{(0)} + h_1 \mathbf{x}^{(1)} + \dots + h_K \mathbf{x}^{(K)} \quad (41)$$

5.4.1 Implementazione

```

1 def filter(self, x: np.ndarray) -> np.ndarray:
2
3     L = self.topology.L_norm_sparse
4
5     # Passo k = 0: segnale di base pesato per h_0
6     curr = x.copy()
7     y = self.h[0] * curr
8
9     # Calcolo ricorsivo dei k-hop successivi: x^(k) = L * x^(k-1)
10    for k in range(1, self.K + 1):
11        curr = L.dot(curr)
12        y += self.h[k] * curr
13
14    return y

```

6 Denoising spettrale basato su stima empirica dell'energia

6.1 Risposta sui singoli vertici e analisi spettrale

La Figura 1 illustra il filtraggio condotto su un grafo Geometrico Casuale con $N = 50$ nodi, ordine polinomiale $K = 6$, regolarizzazione $\mu = 10^{-5}$ e rumore in ingresso $\text{SNR}_{\text{in}} = 7.1$ dB.

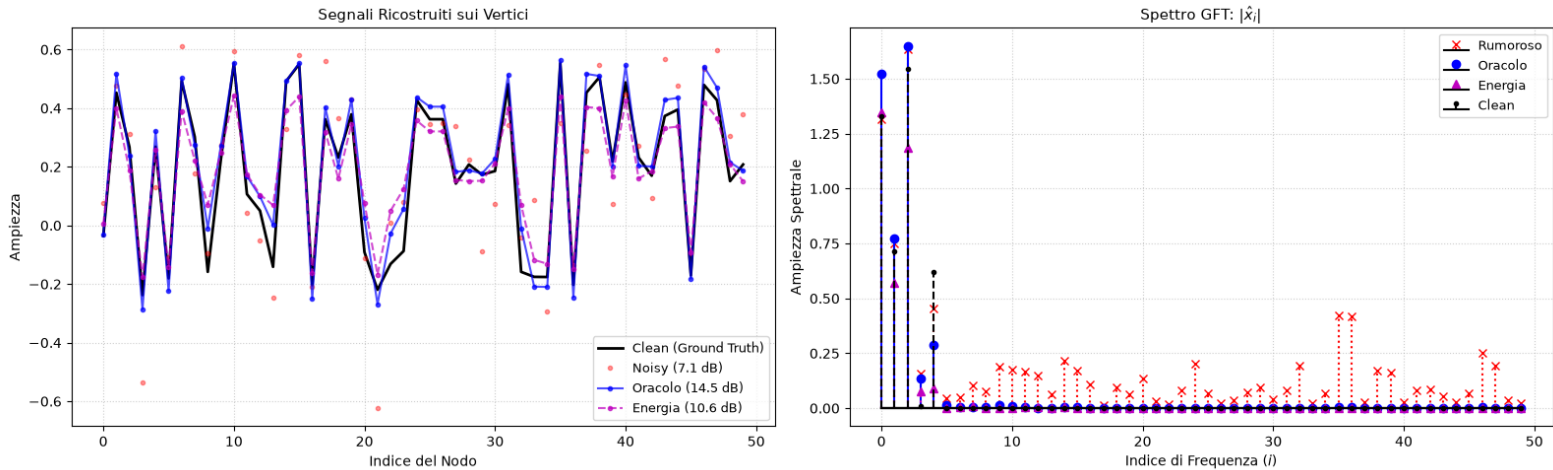


Figura 1: Confronto sul singolo shot di simulazione ($\text{SNR}_{\text{in}} = 7.1$ dB, $K = 6$). Sinistra: ricostruzione sui vertici. Destra: spettro GFT $|\hat{x}_i|$ per i diversi approcci.

Dall'analisi qualitativa e quantitativa emergono i seguenti comportamenti:

- **Filtraggio delle componenti ad alta frequenza:** Nello spettro GFT (Figura 1, destra), il segnale rumoroso presenta coefficienti non nulli su tutto lo spettro ($i > 5$).

Sia il filtro Oracolo sia il filtro ad Energia azzerano efficacemente i modi spettrali $i \geq 6$, rigettando la maggior parte della potenza del rumore.

- **Guadagno di SNR:** Il filtro Oracolo, disponendo della soglia esatta ($i_{\text{cutoff}} = 5$), raggiunge un $\text{SNR}_{\text{out}} = 14.5$ dB (guadagno di +7.4 dB). Il metodo ad Energia, basato sulla soglia al 75%, converge a $\text{SNR}_{\text{out}} = 10.6$ dB, garantendo comunque un incremento netto di +3.5 dB senza alcuna conoscenza a priori del segnale.
- **Distribuzione Spaziale sui Vertici:** La ricostruzione topologica 2D (Figura 2) conferma che le perturbazioni cromatiche presenti nel segnale degradato vengono smussate.

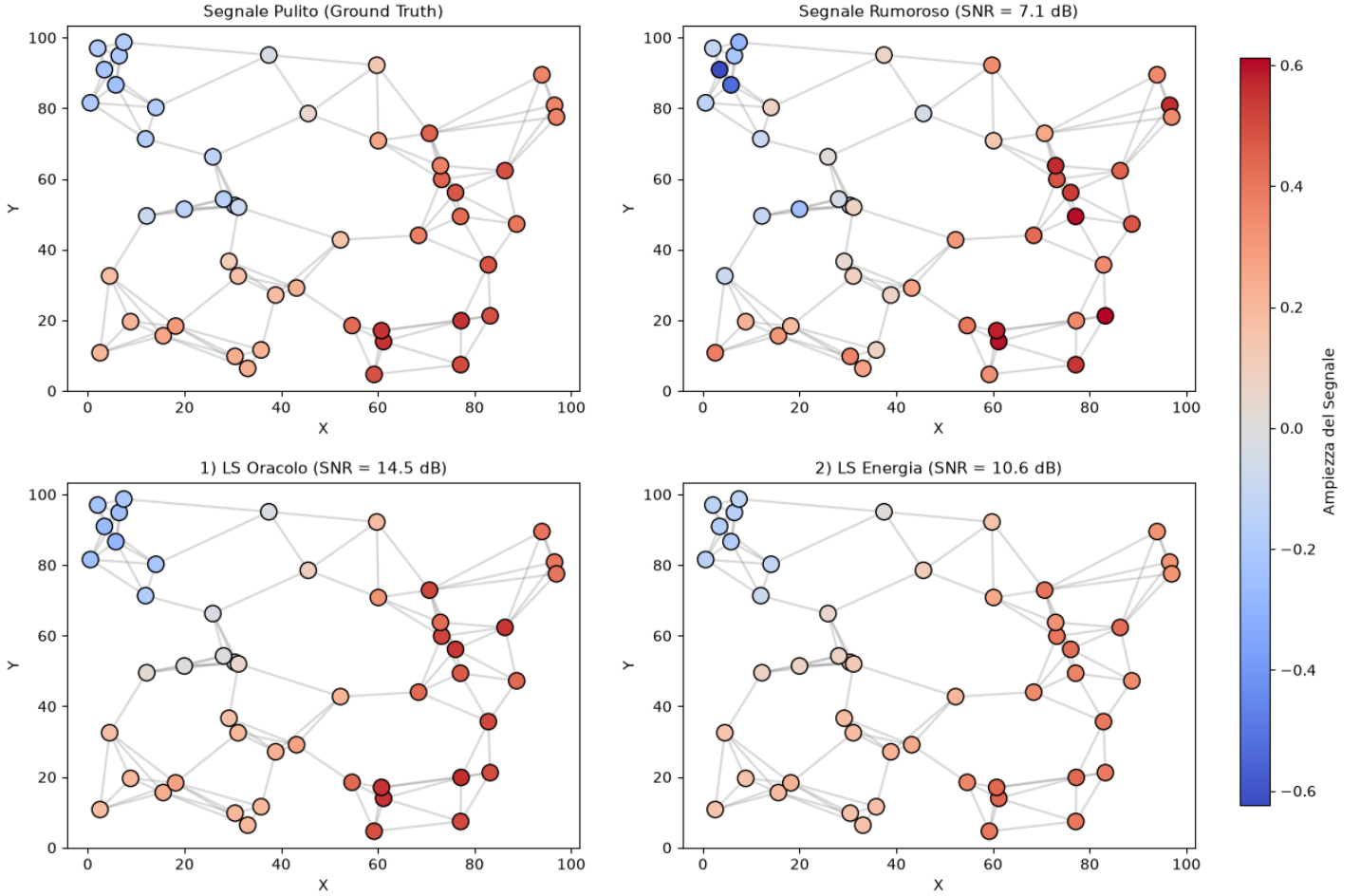
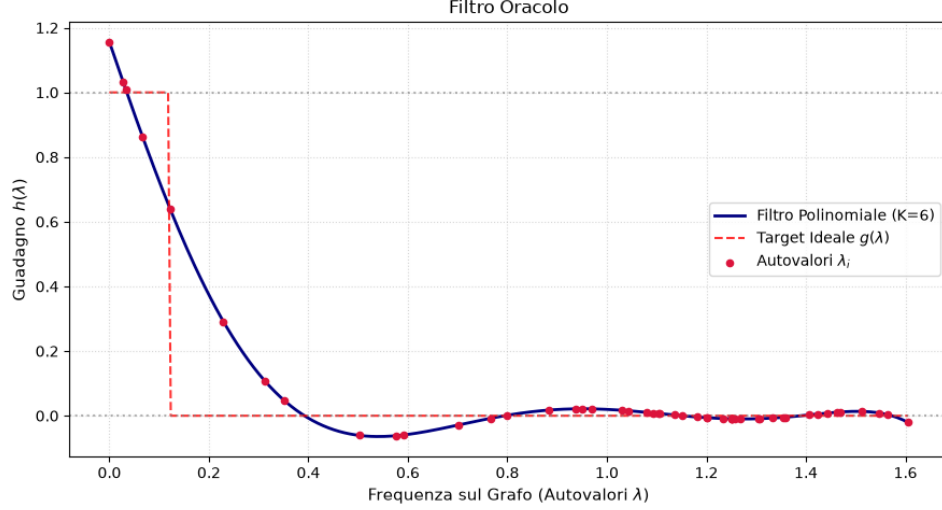
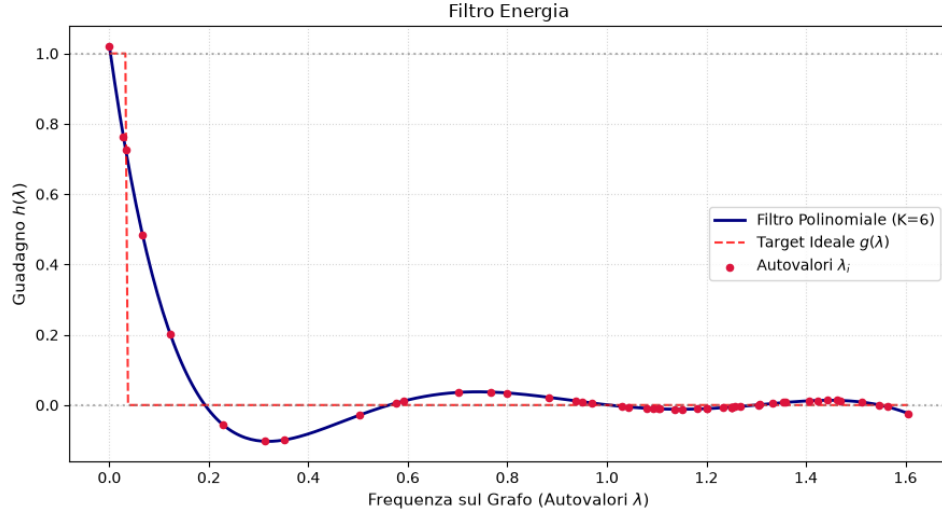


Figura 2: Distribuzione topologica del segnale sui nodi del grafo: Ground Truth, segnale rumoroso ($\text{SNR} = 7.1$ dB), stima Oracolo (14.5 dB) e stima Energia (10.6 dB).

La Figura 3 mette a confronto la risposta in frequenza $h(\lambda) = \sum_{k=0}^K h_k \lambda^k$ fittata rispetto alle rispettive funzioni target ideali $g(\lambda)$.



(a) Filtro Oracolo ($\lambda_c \approx 0.12$)



(b) Filtro Energia ($\lambda_c \approx 0.04$)

Figura 3: Risposta in frequenza $h(\lambda)$ dei filtri polinomiali di ordine $K = 6$. La curva continua blu rappresenta il polinomio continuo, la linea tratteggiata rossa il target ideale a gradino e i punti rossi gli autovalori discreti λ_i del grafo.

Dal confronto visivo tra la Figura 3a e la Figura 3b si nota chiaramente che l'azione del filtro ad Energia è fortemente condizionata dalla concentrazione di potenza sui primissimi modi a frequenza quasi nulla. La soglia viene quindi saturata prematuramente (già al secondo autovalore), provocando uno spostamento verso sinistra della frequenza di taglio. Di conseguenza, il target del filtro ad energia impone a zero il guadagno per armoniche utili intermedie ($i \in [2, 4]$), introducendo una parziale distorsione del segnale originale che giustifica il minor incremento di SNR rispetto all'oracolo ideale (10.6 dB contro 14.5 dB).

6.1.1 Analisi di robustezza tramite grid search

Per valutare la sensibilità del metodo al variare dell'ordine polinomiale $K \in \{2, 4, 6, 8, 12\}$ e dell' $\text{SNR}_{\text{in}} \in [0, 20]$ dB, è stata condotta una simulazione a griglia i cui risultati sono riportati in Figura 4.

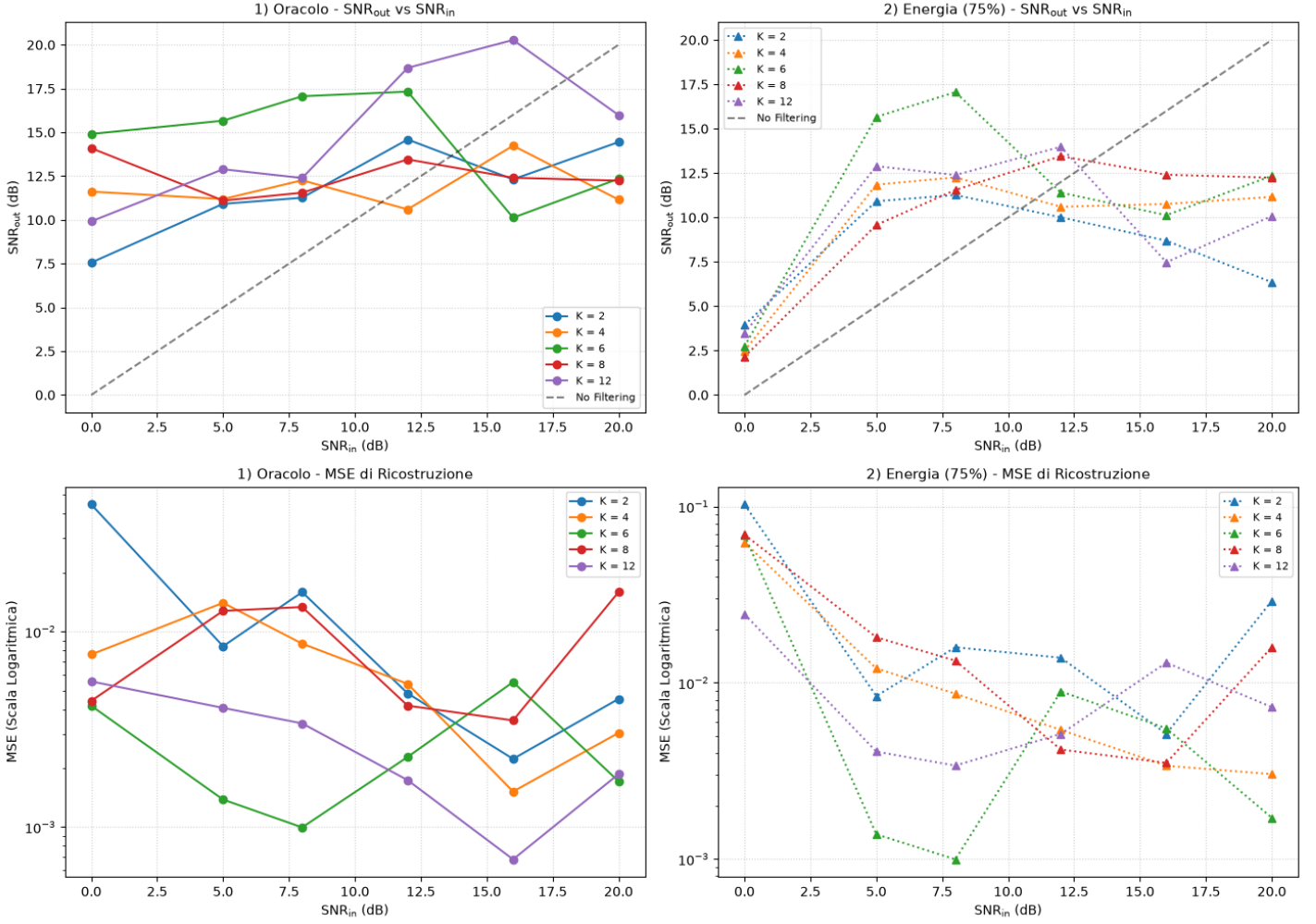


Figura 4: Grid Search per Filtro Oracolo (sinistra) e Filtro ad Energia al 75% (destra).
Riga 1: SNR_{out} vs SNR_{in} . Riga 2: MSE di ricostruzione in scala semilogaritmica.

1. **Regione di guadagno per rumore elevato ($SNR_{in} \leq 12$ dB):** Il metodo ad energia si colloca stabilmente sopra la diagonale di riferimento (*No Filtering*). In particolare per ordini $K \in \{6, 8, 12\}$, il filtro raggiunge picchi di $SNR_{out} \approx 17$ dB per $SNR_{in} = 8$ dB, offrendo una soppressione del rumore quasi paragonabile all'oracolo.
2. **Degradazione a bassi livelli di rumore ($SNR_{in} \geq 16$ dB):** Quando il segnale di partenza è già molto pulito, la soglia euristica al 75% tende a tagliare una frazione residua di energia appartenente alla banda base utile, portando la curva al di sotto della retta *No Filtering*.
3. **Compromesso sull'ordine polinomiale K :** Gli ordini bassi ($K = 2$) soffrono di transizioni spettrali troppo blande, mentre ordini intermedi ($K = 6, 8$) realizzano il miglior trade-off.

7 Denoising guidato dai dati (Data-Driven System Identification)

7.1 Risposta sui singoli vertici e analisi spettrale

La procedura è stata validata su un grafo Geometrico Casuale con $N = 50$ nodi, fissando $K = 6$, $\mu = 10^{-3}$ e generando un segnale a banda limitata ($i \leq 5$) con disturbo d'ingresso $\text{SNR}_{\text{in}} = 8.3$ dB.

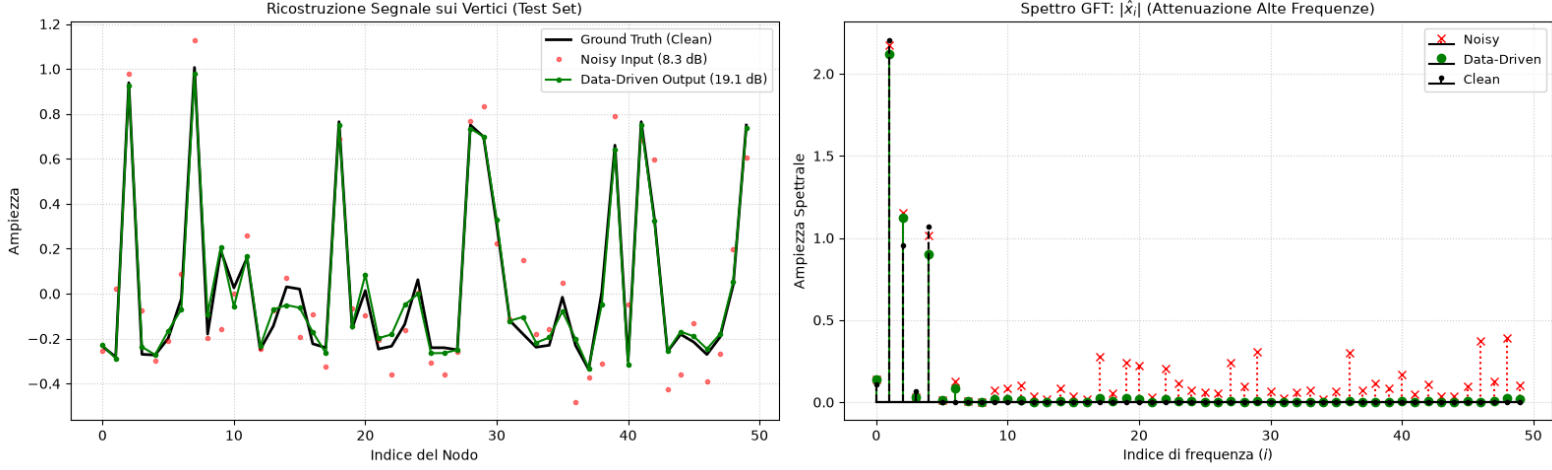


Figura 5: Prestazioni del filtro Data-Driven sul Test Set ($\text{SNR}_{\text{in}} = 8.3$ dB, $K = 6$, $\mu = 10^{-3}$). Sinistra: ricostruzione del segnale sui vertici. Destra: spettro GFT $|\hat{x}_i|$.

Dall'ispezione della Figura 5 e dalla rappresentazione topologica in Figura 6:

- **Soppressione nello spettro GFT:** Nello spettro trasformato (Figura 5, destra), il rumore ad alta frequenza ($i \geq 6$) risulta pressoché azzerato, mentre le prime 5 componenti modali utili seguono con elevata precisione il profilo della ground truth pulita.
- **Coerenza spaziale sui nodi:** La distribuzione dell'ampiezza sui vertici del grafo (Figura 6) mostra come le oscillazioni puntuali del rumore vengano rimosse, restituendo una configurazione cromatica coincidente con il segnale originale.

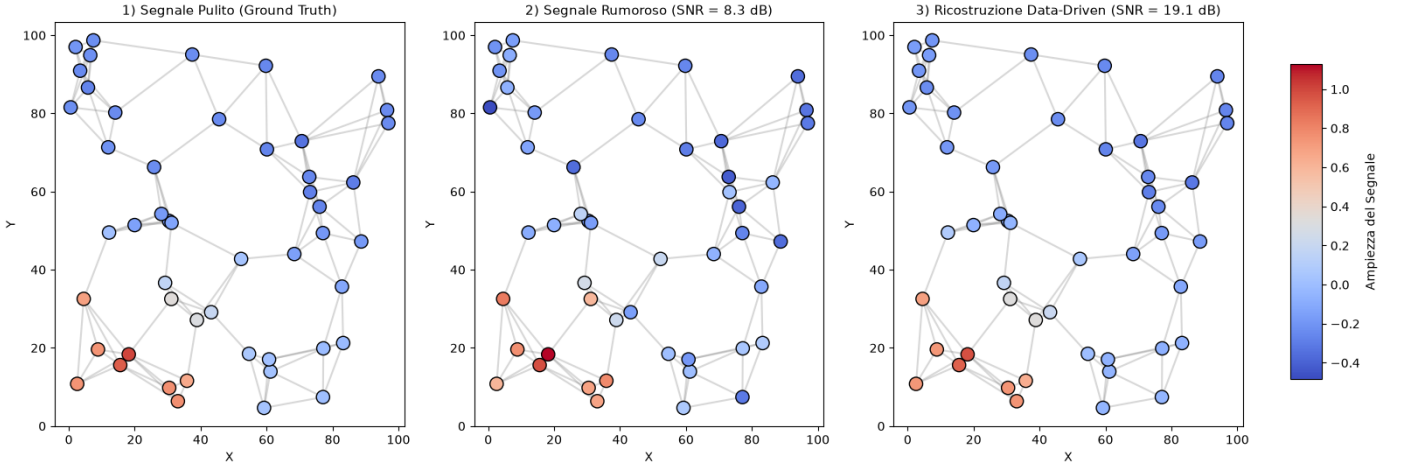


Figura 6: Confronto topologico sul grafo 2D: Ground Truth pulita (sinistra), segnale di test rumoroso a 8.3 dB (centro) e ricostruzione ottenuta dal filtro Data-Driven a 19.1 dB (destra).

La Figura 7 illustra la risposta in frequenza $h(\lambda) = \sum_{k=0}^K h_k \lambda^k$ appresa dal modello data-driven con ordine polinomiale $K = 6$ e parametro di regolarizzazione $\mu = 10^{-3}$.

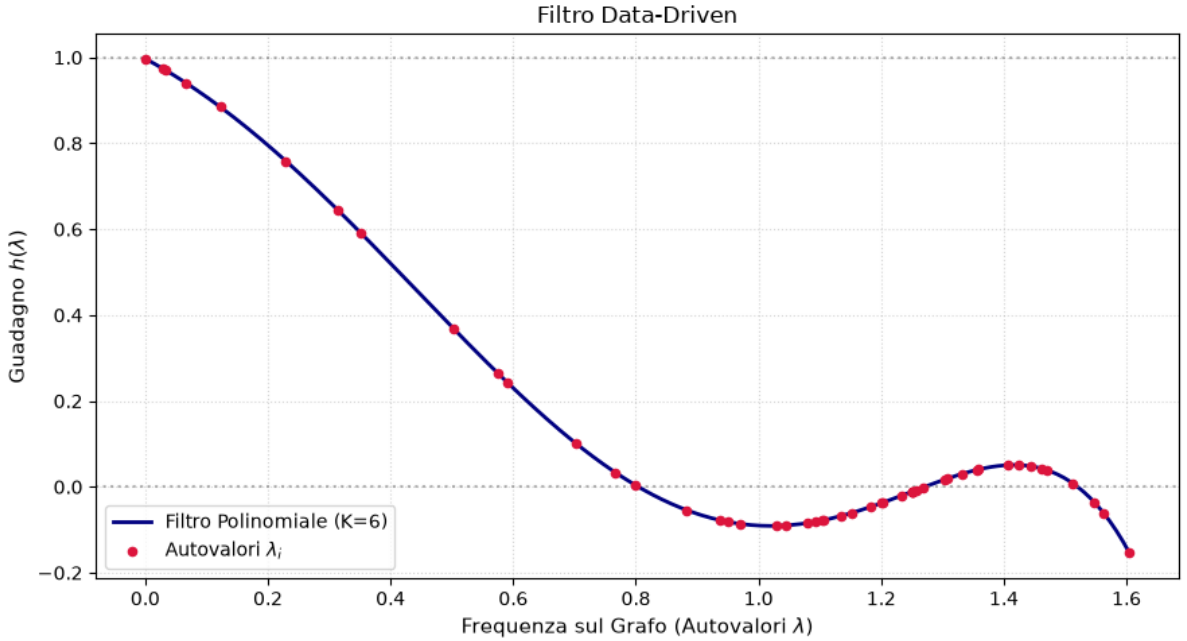


Figura 7: Risposta in frequenza $h(\lambda)$ del filtro data-driven ($K = 6$). La curva blu mostra il profilo continuo del polinomio appreso tramite regressione su sottospazi di Krylov, mentre i punti rossi evidenziano la risposta valutata sui reali autovalori λ_i del grafo.

7.1.1 Analisi di generalizzazione al variare di K e SNR_{in}

Per quantificare la robustezza e la stabilità del metodo al variare del livello di rumore e della complessità del modello, è stata eseguita una Grid Search valutata esclusivamente sul set di test per $K \in \{2, 4, 6, 8, 12\}$ e $\text{SNR}_{\text{in}} \in [0, 20]$ dB (Figura 8).

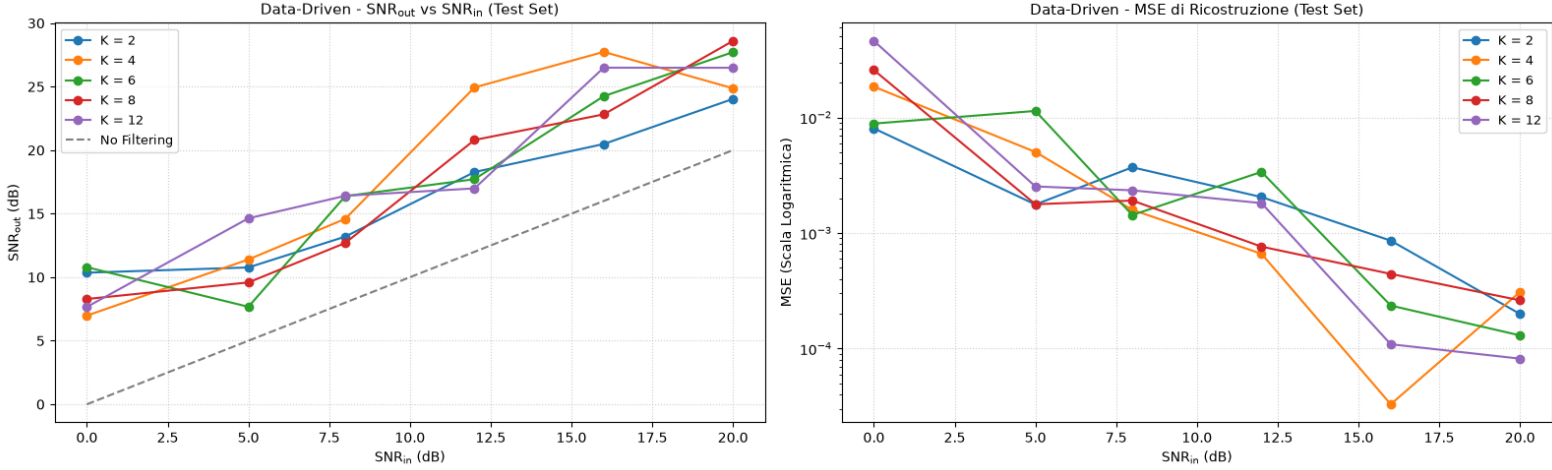


Figura 8: Grid Search Data-Driven sul Test Set. Sinistra: SNR_{out} in funzione di SNR_{in} rispetto alla retta di riferimento (*No Filtering*). Destra: Errore Quadratico Medio (MSE) di ricostruzione in scala semilogaritmica.

Dall'analisi parametrica emergono le seguenti proprietà:

1. **Guadagno sistematico sul test set:** Per l'intero intervallo di rumore esplorato, tutte le curve di SNR_{out} rimangono nettamente al di sopra della retta (*No Filtering*). A differenza del metodo spettrale ad energia (che mostrava saturazione ad alti SNR), l'approccio data-driven continua a incrementare la qualità di ricostruzione, superando i 25–28 dB per $\text{SNR}_{\text{in}} = 20$ dB. Tale progressione è favorita dal fatto che, nel metodo di simulazione adottato, il modello viene calibrato al variare di SNR_{in} , consentendo ai coefficienti $\mathbf{h}$ di adattarsi dinamicamente all'effettivo rapporto segnale-rumore presente nel canale.

Il loop di addestramento e validazione è schematizzato come segue:

- **Inizializzazione:** Si fissa la matrice Laplaciana normalizzata $\mathbf{L}$, il parametro di regolarizzazione μ , l'insieme degli ordini polinomiali $\mathcal{K}$ e la griglia dei livelli di rumore $\mathcal{S}_{\text{in}}$.
- **Ciclo esterno** ($K \in \mathcal{K}$): Itera sulla dimensione del sottospazio di Krylov (ordine del filtro polinomiale).
- **Ciclo interno** ($\text{SNR}_{\text{in}} \in \mathcal{S}_{\text{in}}$): Itera sul livello di rumore additivo AWGN target:
 - **Campionamento train:** Genera un segnale a banda limitata $\mathbf{x}_{\text{clean}}^{\text{train}}$ e la corrispondente versione rumorosa $\mathbf{x}_{\text{noisy}}^{\text{train}}$.
 - **Campionamento test:** Genera una seconda realizzazione indipendente $\mathbf{x}_{\text{clean}}^{\text{test}}$ e la relativa istanza perturbata $\mathbf{x}_{\text{noisy}}^{\text{test}}$.
 - **Stima supervisionata:** Costruisce la matrice dei segnali diffusi:

$$\mathbf{X} = [\mathbf{x}_{\text{noisy}}^{\text{train}}, \mathbf{L}\mathbf{x}_{\text{noisy}}^{\text{train}}, \dots, \mathbf{L}^K \mathbf{x}_{\text{noisy}}^{\text{train}}]$$

e calcola il vettore dei coefficienti ottimi:

$$\mathbf{h}^* = (\mathbf{X}^\top \mathbf{X} + \mu \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{x}_{\text{clean}}^{\text{train}}$$

- **Inferenza e metriche:** Applica il filtro $\mathbf{H}(\mathbf{L}) = \sum_{k=0}^K h_k^* \mathbf{L}^k$ sul segnale di test $\mathbf{x}_{\text{noisy}}^{\text{test}}$ e memorizza le metriche prestazionali di SNR_{out} ed MSE.
- 2. **Comportamento dell'errore quadratico medio (MSE):** In scala semilogaritmica, l'MSE decresce linearmente con l'aumento dell' SNR_{in} , per la motivazione vista nel punto precedente.

8 Riproducibilità

La generazione della topologia del grafo, la sintesi dei segnali a banda limitata e la modellazione del rumore additivo dipendono da processi pseudo-casuali (`numpy.random`). Per garantire la riproducibilità, l'inizializzazione del generatore è stata controllata mediante i *seeds* dedicati per ciascuna fase:

- **Riproducibilità sperimentale:** Fissando il seed iniziale per la topologia (`seed=42`), la disposizione spaziale dei vertici $\mathbf{p}_i \in [0, 100]^2$, le connessioni di vicinato k -NN e la conseguente struttura spettrale del Laplaciano normalizzato $\mathbf{L}_{\text{norm}}$ rimangono deterministiche e replicabili in ogni sessione di simulazione.
- **Disgiunzione rigorosa tra training e test (*Data Leakage Prevention*):** Nel paradigma supervisionato *data-driven* (trattato in dettaglio nella Sezione 7), è fondamentale evitare che il filtro memorizzi il profilo del disturbo della singola realizzazione, perciò le coppie di segnali $(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{noisy}})$ per il training e il test sono generate con semi diversi.

Riferimenti bibliografici

- [1] E. Isufi, F. Gama, D. I. Shuman, and S. Segarra, “Graph filters for signal processing and machine learning on graphs,” *arXiv preprint arXiv:2211.08854*, 2024.